

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»
ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИKN)
КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)

Курс «Клиент-серверные приложения и сетевые технологии»

Пояснительная записка к курсовой работе

на тему:

«Разработка клиент-серверного приложения Toy Shelf для учета игрушек»

Выполнил: студент группы БИВТ-24-4

Кадыров Артур Рустамович

Подпись: _____

Проверили: Абросимов Никита Андреевич

Шелудяков Пётр Алексеевич

Москва, 2026

Оглавление

1. Предметная область	2
2. Постановка задачи	3
3. Описание архитектуры	4
4. Описание структуры БД	5
5. Описание серверной части	6
6. Описание клиентской части	7
7. Заключение	8
8. Список литературы	9

1. Предметная область

Предметной областью курсовой работы является разработка персонального клиент-серверного веб-приложения для учета игрушек и пользователей системы. Приложение Toy Shelf демонстрирует типовой сценарий работы небольшого интернет-магазина игрушек: пользователь открывает клиентскую часть в браузере, проходит демонстрационную авторизацию, просматривает каталог, фильтрует товары по категориям, добавляет игрушки в корзину и видит итоговую сумму заказа.

Тема «игрушка» выбрана как простая и наглядная предметная область. Игрушка в приложении выступает объектом каталога, который имеет название, категорию, цену и признак наличия. Пользователь выступает субъектом системы и содержит идентификатор, имя, адрес электронной почты и необязательный возраст. Такое разделение позволяет показать и предметную часть интерфейса, и обязательную часть задания, связанную с API пользователей.

Проблема предметной области состоит в необходимости централизованно получать и проверять данные, а также ограничивать действия пользователей по категориям доступа. В простом приложении без валидации пользователь может отправить пустое имя или некорректный email, из-за чего данные становятся непригодными для дальнейшей обработки. Без разделения доступа любой посетитель мог бы выполнять административные действия. Поэтому в приложении предусмотрены серверная проверка входных данных, обработка ошибок и клиентское управление доступом.

Приложение ориентировано на учебную демонстрацию возможностей курса: HTML и CSS применяются в клиентском интерфейсе, LocalStorage используется для сохранения демо-сессии, REST API реализует обмен между клиентом и сервером, а NestJS показывает работу контроллеров, сервисов, провайдеров, декораторов, DTO и фильтров исключений.

2. Постановка задачи

Цель работы — разработать персональное клиент-серверное приложение Toy Shelf, проверить его работоспособность и подготовить пояснительную записку по выполненной работе. Приложение должно демонстрировать основные изученные возможности: HTML, CSS, API, LocalStorage, Git, управление доступом по категориям клиентов, обработку HTTP-запросов и валидацию данных.

В серверной части необходимо реализовать REST API для работы с пользователями. Обязательные endpoint: GET /users — получение списка всех пользователей, GET /users/:id — получение пользователя по идентификатору, POST /users — создание нового пользователя. Каждый пользователь содержит поля id, name, email и age. Поля id, name и email являются обязательными, поле age является необязательным.

Поле email должно проходить проверку формата адреса электронной почты. Для этого в проекте используется библиотека class-validator. Входные данные проверяются через DTO и глобальный ValidationPipe, что позволяет отклонять некорректные запросы до попадания данных в сервисную логику.

По индивидуальной постановке задания данные пользователей должны храниться в памяти и не должны сохраняться между перезапусками приложения. Поэтому в проекте не используется постоянная база данных: массив пользователей хранится внутри UsersService. Это решение соответствует специальному требованию о временном хранении данных. В PDF с общими требованиями к курсовой работе приведены расширенные требования к БД; для данной версии задания они не применяются, поскольку противоречат условию о хранении данных в памяти.

Дополнительно реализованы endpoint POST /auth/login для демонстрации категорий доступа и GET /toys для вывода каталога игрушек. На клиентской стороне реализована корзина, которая сохраняется в LocalStorage. Эти возможности не заменяют обязательные endpoint /users, а расширяют приложение и позволяют показать более полноценный пользовательский сценарий.

3. Описание архитектуры

Toy Shelf построен по клиент-серверной архитектуре. Клиентская часть реализована на React и запускается через Vite. Серверная часть реализована на NestJS и запускается как отдельное HTTP-приложение. Клиент и сервер взаимодействуют по протоколу HTTP, а данные передаются в формате JSON.

Клиентская часть отвечает за отображение интерфейса, хранение состояния, ввод данных, отправку fetch-запросов, сохранение демо-сессии и корзины в LocalStorage. Серверная часть отвечает за маршрутизацию, валидацию данных, выполнение

бизнес-логики, формирование JSON-ответов, обработку исключений и логирование запросов.

Архитектура сервера разделена на модули. `UsersModule` отвечает за пользователей, `AuthModule` — за демонстрационную авторизацию и категории доступа, `ToysModule` — за каталог игрушек. Такое разделение упрощает сопровождение проекта: каждая предметная область имеет собственный контроллер и сервис.

Контроллеры принимают HTTP-запросы и извлекают из них параметры. Сервисы выполняют прикладную логику и хранят данные. DTO описывают структуру входных данных и правила проверки. Провайдеры `NestJS` позволяют внедрять сервисы в контроллеры через `dependency injection`.

В приложении используется `CORS`, чтобы `React`-клиент мог обращаться к API `NestJS` во время локальной разработки. Сервер запускается на порту 3000, клиент `Vite` обычно запускается на порту 5173.

Компонент	Назначение
React-клиент	Отображение интерфейса, работа с <code>LocalStorage</code> , отправка HTTP-запросов
NestJS-сервер	REST API, валидация, обработка ошибок и логирование
UsersModule	Работа с пользователями
AuthModule	Демонстрационная авторизация и категории доступа
ToysModule	Демонстрационный каталог игрушек

Таблица 1 — Компоненты архитектуры

4. Описание структуры БД

В данной реализации постоянная база данных отсутствует, потому что индивидуальное задание требует хранить данные в памяти и не сохранять их между перезапусками приложения. Поэтому раздел описывает не физическую БД, а структуру `in-memory` данных, используемых серверной частью.

Основная структура данных — `User`. Она используется обязательными endpoint `/users` и хранится в массиве внутри `UserService`. При создании пользователя сервер самостоятельно генерирует `id` с помощью `randomUUID`. Клиент не передает `id` в `POST`-запросе, что предотвращает конфликт идентификаторов.

Сущность `User` включает поля `id`, `name`, `email` и `age`. Поле `name` хранит имя пользователя, `email` хранит адрес электронной почты, `age` хранит возраст и может отсутствовать. Перед сохранением `email` приводится к нижнему регистру, а `name` очищается от лишних пробелов по краям.

Дополнительная структура `Toy` используется для отображения темы приложения. Она содержит `id`, `title`, `category`, `price`, `available`, `stock`, `rating`, `description` и `image`. Данные игрушек также хранятся в памяти и служат для демонстрации интерфейса магазина, карточек товаров, фильтра категорий и работы клиента с API.

При перезапуске backend-приложения все созданные во время работы пользователи исчезают, а сервис снова инициализируется начальными демонстрационными данными. Это поведение проверяется вручную: создать пользователя, убедиться, что он появился в `GET /users`, перезапустить сервер и повторно открыть `GET /users`.

Поле	Тип	Обязательность	Назначение
<code>id</code>	<code>string</code>	да	Уникальный идентификатор пользователя
<code>name</code>	<code>string</code>	да	Имя пользователя
<code>email</code>	<code>string</code>	да	Адрес электронной почты
<code>age</code>	<code>number</code>	нет	Возраст пользователя

Таблица 2 — Структура сущности `User`

5. Описание серверной части

Серверная часть реализована на NestJS. Входной файл `main.ts` создает приложение через `NestFactory`, включает `CORS`, подключает глобальный `ValidationPipe`, регистрирует фильтр исключений `HttpExceptionFilter` и `middleware` логирования `RequestLoggerMiddleware`.

`AppModule` является корневым модулем приложения. Он подключает `UsersModule`, `AuthModule` и `ToysModule`. Модульная структура соответствует подходу NestJS: каждая область содержит собственный контроллер и сервис, а сервисы регистрируются как провайдеры.

`UsersController` реализует обязательные endpoint задания. Метод `findAll` обрабатывает `GET /users` и возвращает весь массив пользователей. Метод `findOne` обрабатывает `GET /users/:id` и передает `id` в сервис. Метод `create` обрабатывает `POST /users`, принимает DTO из тела запроса и создает пользователя.

`UserService` содержит прикладную логику работы с пользователями. Метод `findAll` возвращает массив, `findOne` ищет пользователя по `id` и выбрасывает `NotFoundException` при отсутствии записи, `create` проверяет уникальность `email` и выбрасывает `ConflictException` при повторе адреса. После проверки создается объект `User` с новым `id`.

`CreateUserDto` описывает входные данные `POST /users`. Для `name` используются `IsString` и `IsNotEmpty`. Для `email` используются `IsEmail` и `IsNotEmpty`. Для `age` используются `IsOptional`, `Type`, `IsInt`, `Min` и `Max`. Благодаря этому сервер отклоняет пустые

обязательные поля, некорректный email, лишние поля и некорректный возраст.

HttpExceptionFilter приводит ошибки к единому виду JSON. Ответ содержит statusCode, timestamp и details. Такой формат удобен для клиента: frontend может получить сообщение ошибки и вывести его пользователю в интерфейсе.

RequestLoggerMiddleware фиксирует метод запроса, URL, HTTP-статус и время выполнения в миллисекундах. Логирование помогает при защите показать, что сервер действительно получает запросы от клиента и корректно отвечает на них.

AuthService реализует демонстрационную авторизацию. Для ролей admin и client используются логины и bcrypt-хеши паролей. Пароли не хранятся в открытом виде. Для guest предусмотрен упрощенный вход без пароля. В ответ клиент получает демонстрационный token, роль и логин.

С точки зрения безопасности важно, что проверка прав на создание пользователя в данной реализации демонстрируется на клиентской стороне. Для реального промышленного приложения нужно дополнительно проверять роль на сервере через guards и JWT. В рамках курсовой это описано как направление развития, а текущая версия показывает управление доступом по категориям клиентов.

Проверка API выполняется через браузер, Postman, Thunder Client или PowerShell. Например, GET `http://localhost:3000/users` возвращает массив пользователей, а POST `http://localhost:3000/users` с телом `{"name":"Test User","email":"test@example.com","age":20}` создает нового пользователя.

Endpoint	Метод	Назначение
/users	GET	Получение списка всех пользователей
/users/:id	GET	Получение пользователя по id
/users	POST	Создание пользователя
/auth/login	POST	Демонстрационная авторизация
/toys	GET	Получение каталога игрушек

Таблица 3 — Основные endpoint приложения

6. Описание клиентской части

Клиентская часть реализована на React и Vite. Главный компонент App.jsx содержит состояние текущей сессии, формы входа, списка пользователей, списка игрушек, формы создания пользователя и сообщения об ошибке. Разметка написана на JSX, а стили вынесены в файл styles.css.

Интерфейс состоит из главного блока с краткой статистикой, формы авторизации, каталога игрушек, фильтра категорий, корзины, списка пользователей и административной формы создания пользователя. Верхняя панель показывает текущую категорию доступа. Если пользователь вошел в систему, рядом отображается кнопка выхода.

Форма входа позволяет выбрать роль `guest`, `client` или `admin`. Для `client` используется логин `client` и пароль `client123`, для `admin` — `admin` и `admin123`. После успешного входа данные сессии сохраняются в `LocalStorage`, поэтому при обновлении страницы состояние входа восстанавливается.

Функции `loadSession`, `saveSession` и `clearSession` находятся в `storage.js`. Они работают с ключом `toy-shelf-session`. Для корзины используются `loadCart`, `saveCart` и `clearCart` с отдельным ключом `toy-shelf-cart`. `LocalStorage` выбран потому, что он изучается в рамках курса и позволяет показать хранение состояния на стороне клиента. При этом в отчете отдельно отмечено, что чувствительные данные нельзя хранить в `LocalStorage` без дополнительных мер защиты.

Функция `request` из `api.js` инкапсулирует работу с `Fetch API`. Она отправляет запрос на сервер, разбирает `JSON`-ответ, проверяет `HTTP`-статус и преобразует ошибки в текстовое сообщение. Благодаря этому `App.jsx` не дублирует обработку ошибок для каждого запроса.

Управление доступом реализовано через условный рендеринг. Гость может просматривать витрину, клиент и администратор могут добавлять товары в корзину и оформлять демонстрационный заказ. Администратор дополнительно видит форму создания пользователя. После отправки формы клиент вызывает `POST /users`, затем обновляет список пользователей через `GET /users`.

`CSS`-файл задает визуальное оформление приложения: сетку контента, панели, элементы списков, формы, кнопки и адаптивное поведение. На экранах небольшой ширины двухколоночная сетка перестраивается в одну колонку, поэтому интерфейс остается читаемым на ноутбуках и мобильных устройствах.

Для отчета рекомендуется приложить скриншоты: главный экран приложения, карточки товаров с изображениями, фильтр категорий, корзину с итоговой суммой, вход под клиентом, вход под администратором, форму создания пользователя, успешное добавление пользователя, ошибку валидации `email`, ответ `GET /users`, ответ `GET /toys` и лог запросов `backend` в терминале.

7. Заключение

В ходе курсовой работы разработано клиент-серверное приложение `Toy Shelf`. Приложение реализует обязательные `endpoint` для работы с пользователями, хранит

данные в памяти, выполняет валидацию email и обязательных полей, обрабатывает ошибки и возвращает JSON-ответы.

Серверная часть демонстрирует изученные возможности NestJS: модули, контроллеры, сервисы, провайдеры, декораторы, DTO, ValidationPipe, фильтр исключений и middleware логирования. Клиентская часть демонстрирует HTML/JSX, CSS, Fetch API, LocalStorage, витрину товаров, корзину и управление доступом по категориям пользователей.

Поставленная задача выполнена в соответствии с индивидуальной постановкой: данные не сохраняются между перезапусками, API пользователей реализовано, обязательные поля проверяются, email валидируется, а приложение может быть запущено локально и проверено через браузер.

Возможные направления развития проекта: перенос хранения данных из памяти в PostgreSQL, добавление серверных guards для проверки ролей, использование JWT, добавление полноценного CRUD для игрушек и автоматические тесты для контроллеров и сервисов.

8. Список литературы

- [1] Официальная документация NestJS. URL: <https://docs.nestjs.com/>
- [2] Официальная документация React. URL: <https://react.dev/>
- [3] Документация Vite. URL: <https://vite.dev/>
- [4] Документация MDN Web Docs по Fetch API. URL: https://developer.mozilla.org/docs/Web/API/Fetch_API
- [5] Документация MDN Web Docs по LocalStorage. URL: <https://developer.mozilla.org/docs/Web/API/Window/localStorage>
- [6] Документация class-validator. URL: <https://github.com/typestack/class-validator>
- [7] Документация bcryptjs. URL: <https://www.npmjs.com/package/bcryptjs>
- [8] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления.
- [9] Репозиторий исходного кода приложения Toy Shelf. URL: <https://github.com/Artlogg/toy-shelf-coursework>